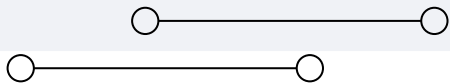


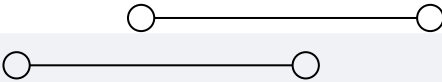


Paralelização de Regressão Logística com OpenMP



Grupo: Augusto Kessler Pires (00338627) e Vitor Chassot (00302134)





Regressao Logística

- Modelo de classificação binário
- Calcula combinação linear das entradas
- Aplica a função sigmoide para obter probabilidade
- Atualiza parâmetros via gradiente descendente

$$Z = X \cdot W + b$$

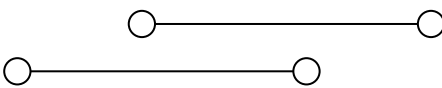
$$p_i = \sigma(Z_i) = \frac{1}{1 + e^{-Z_i}}$$

$$\text{grad}[0] = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)$$

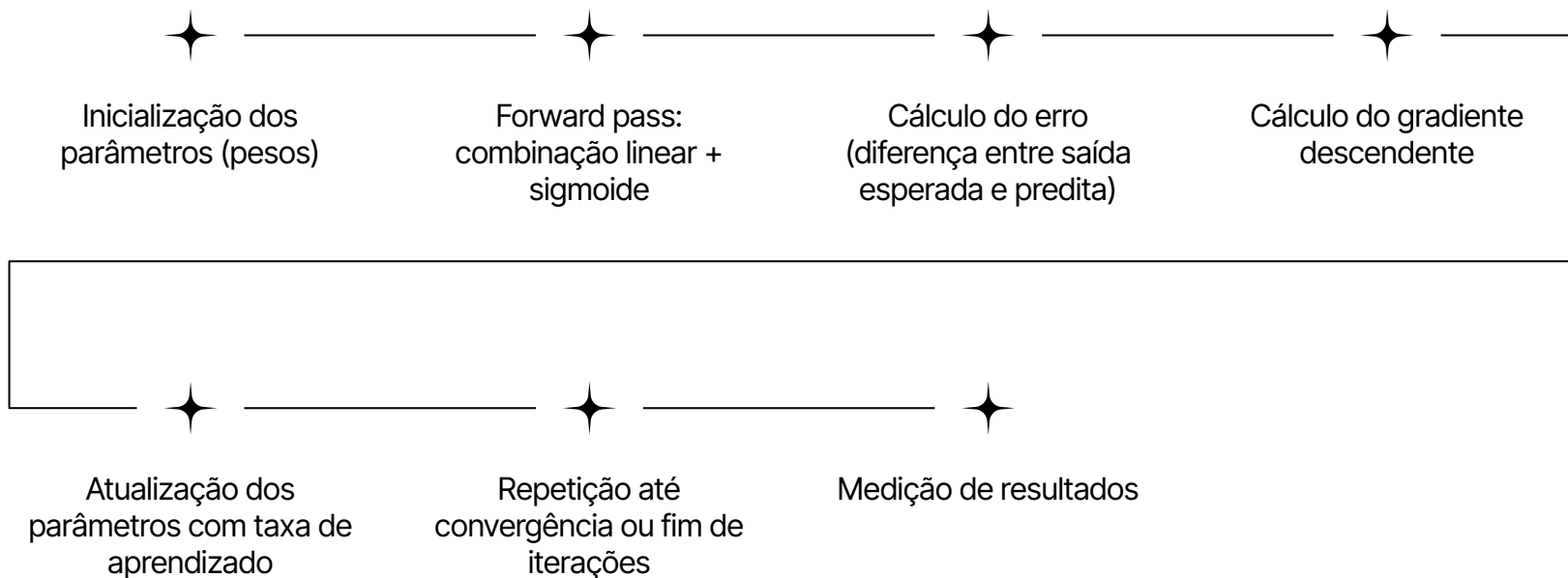
$$\text{grad}[j] = \frac{1}{N} \sum_{i=1}^N (p_i - y_i) \cdot X_{ij}, \quad j = 1..P$$

$$B_j := B_j - \alpha \cdot \text{grad}_j, \quad j = 0..P$$



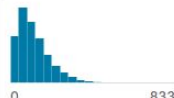


Funcionamento da regressão logística



Dataset utilizado

- Diabetes Health Indicator Data (obtido do Kaggle)
- Classificação binária: `diagnosed_diabetes`: 0 = Não, 1 = Sim
- Dados gerados usando distribuições estatísticas inspiradas por pesquisas médicas da vida real
- 100.000 Linhas e 35 Atributos, com atributos numéricos e categóricos

# age	Δ gender	Δ smoking_status	# physical_activity....	# diagnosed_dia
Age of patient (18-90 years)	Male, Female, Other	Smoking behavior (Never, Former, Current)	Weekly physical activity minutes	Binary target (0 : Yes)
	Female 50% Male 48% Other (2013) 2%	Never 60% Current 20% Other (20011) 20%		
18			0	0
58	Male	Never	215	1
48	Female	Former	143	0
60	Male	Never	57	1
74	Female	Never	49	1
46	Male	Never	109	1
46	Female	Never	124	0
75	Female	Never	53	0
62	Male	Current	75	1
42	Male	Current	114	1
59	Female	Current	86	0
43	Female	Never	118	1
43	Female	Former	167	1



Primeiro Código

O código em C implementa:

- Leitura do dataset (CSV)
- Pré-processamento dos dados
- Combinacao linear, sigmoide, cálculo do erro e do gradiente e atualização dos parâmetros
- Cálculo das métricas e impressão de dados

```
int max_iter = atoi(argv[1]);
double X[N][P];
double Y[N];
double Z[N];
double p[N];
double B[P+1];
double gradJ_B[P+1]={0};
iniciar_parametros(B,0.1);//
iniciar_dados_de_treino(X,Y);//
double start, end;
start = omp_get_wtime();
for (int i=0;i<max_iter;i++){//
    combinacao_linear(X,B,Z);
    sigmoide(p,Z);
    gradiente(gradJ_B,p,Y,X);
    double alfa=0.1;
    atualizar_parametros(B,gradJ_B,alfa);//
}
end = omp_get_wtime();
imprimir_parametros(B);//
double Xteste[Nnew][P];
double Yteste[Nnew];
iniciar_dados_de_teste(Xteste,Yteste);//
imprimir_resultados(Xteste,B,Yteste);//
```

Primeiro Código

Funções `combinacao_linear`, `sigmoide`,
`gradiente` e `atualizar_parametros`

```
void combinacao_linear(double X[N][P], double B[P+1], double Z[N]){
    for(int i=0; i<N; i++){
        Z[i]=B[0];
        for(int j=0; j<P; j++){
            Z[i]+=((B[j+1]))*X[i][j];
        }
    }
}

void sigmoide(double p[N], double Z[N]){
    for(int i=0; i<N; i++){
        p[i]=1.0/(1.0+exp(-Z[i]));
    }
}

void gradiente(double gradJ_B[P+1], double p[N], double Y[N], double X[N][P]){
    double grad[P+1];
    for (int j = 0; j <= P; ++j) grad[j] = 0.0;
    for (int i = 0; i < N; ++i) {
        double err = p[i] - Y[i];
        grad[0] += err; // bias
        for (int j = 0; j < P; ++j)
            grad[j+1] += err * X[i][j]; // itera j contiguamente
    }
    for (int j = 0; j <= P; ++j)
        gradJ_B[j] = grad[j] / (double)N;
}

void atualizar_parametros(double B[P+1], double gradJ_B[P+1], double alfa){
    for( int j=0; j<P+1; j++){
        B[j]-=alfa*gradJ_B[j];
    }
}
```



Identificação de Pontos Paralelizáveis



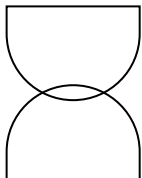
Vale a pena paralelizar

- combinacao_linear (N grande)
- sigmoide (N grande)
- gradiente (loops internos com redução)



Não vale a pena paralelizar:

- Atualizar_parametros (Número de colunas pequeno)
- Funções de impressão (I/O)
- Leitura de dados (I/O bound)



Estratégias de paralelização

Basta inserir **parallel for** nas funções?

- **Criação e destruição de threads:** overhead ocorre a cada etapa.
- **Sincronização frequente:** barreiras implícitas em cada loop aumentam espera entre threads.
- **Reuso de buffers locais:** buffers precisam ser alocados/desalocados a cada chamada, aumentando custo.
- **Granularidade do paralelismo:** loops curtos podem não compensar o overhead de paralelização.

```
void combinacao_linear(double X[N][P], double B[P+1], double Z[N]){
    #pragma omp parallel for
    for(int i=0; i<N; i++){
        Z[i]=B[0];
        for(int j=0; j<P; j++){
            Z[i]+=(B[j+1])*X[i][j];
        }
    }
}

void sigmoide(double p[N], double Z[N]){
    #pragma omp parallel for
    for(int i=0; i<N; i++){
        p[i]=1.0/(1.0+exp(-Z[i]));
    }
}

void gradiente(double gradJ_B[P+1], double p[N], double Y[N], double X[N][P]){
    double grad[P+1];
    for (int j = 0; j <= P; ++j) grad[j] = 0.0;
    #pragma omp parallel for
    for (int i = 0; i < N; ++i) {
        double err = p[i] - Y[i];
        grad[0] += err; // bias
        for (int j = 0; j < P; ++j)
            grad[j+1] += err * X[i][j]; // itera j contigualmente
    }
    for (int j = 0; j <= P; ++j)
        gradJ_B[j] = grad[j] / (double)N;
}
```


Novo Código

O código organiza de forma diferente os loops. Todas as etapas são realizadas em um for para cada linha dos dados (exceto atualização dos pesos devido ao modo batch)

A função sigmoide também é alterada para evitar overflow em caso de números muito negativos.

```
// Função sigmoid estável numericamente
double sigmoid(double z) {
    if (z >= 0) {
        double ez = exp(-z);
        return 1.0 / (1.0 + ez);
    } else {
        double ez = exp(z);
        return ez / (1.0 + ez);
    }
}
```

```
for (int iter = 0; iter < NUM_ITER; iter++) {
    double grad[NUM_FEATURES] = {0.0};
    dbias = 0.0;
    for (int i = 0; i < num_samples; i++) {
        double z = bias; // soma do bias
        for (int j = 0; j < NUM_FEATURES; j++) {
            z += w[j] * X[i][j]; // soma ponderada dos features
        }
        double pred = sigmoid(z); // função de ativação
        double err = pred - y[i]; // diferença predito - real

        for (int j = 0; j < NUM_FEATURES; j++) {
            grad[j] += err * X[i][j]; // gradiente dos pesos
        }
        dbias += err; // gradiente do bias
    }
    for (int j = 0; j < NUM_FEATURES; j++) {
        w[j] -= lr * grad[j] / num_samples; // atualização dos pesos
    }
    bias -= lr * dbias / num_samples; // atualização do bias
}
```

Código Paralelo

Funcionamento da nova região paralela

- **Criação da região paralela:** `#pragma omp parallel` cria todas as threads **uma única vez** para todo o batch de treino. O número de iterações não afetou os resultados das métricas, sendo utilizado principalmente para multiplicar o tempo da execução e evitar problemas de medidas de valores muito baixos.
- **Loop de iterações do treino:** Cada thread calcula **combinação linear**, aplica **sigmoide** e acumula gradientes **no seu buffer local**.
- **Redução segura e atualização de parâmetros** `reduction(+:grad[:NUM_FEATURES],dbias)`: Ao final as variáveis `dbias` e `grad` de cada thread são somadas e usados para atualizar os pesos.
- **Reuso e desalocação:** O buffer local permanece alocado durante todas as iterações do batch e é liberado após o fim desse loop.

```
double start = omp_get_wtime();
for (int iter = 0; iter < NUM_ITER; iter++) {
    double grad[NUM_FEATURES] = {0.0};
    dbias = 0.0;
    #pragma omp parallel for reduction(+:grad[:NUM_FEATURES], dbias)
    for (int i = 0; i < num_samples; i++) {
        double z = bias; // soma do bias
        for (int j = 0; j < NUM_FEATURES; j++)
            z += w[j] * x[i][j]; // soma ponderada dos features
        double pred = sigmoid(z); // função de ativação
        double err = pred - y[i]; // diferença predito - real

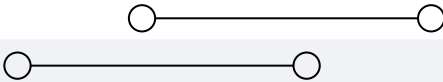
        for (int j = 0; j < NUM_FEATURES; j++)
            grad[j] += err * x[i][j]; // gradiente dos pesos

        dbias += err; // gradiente do bias
    }
    for (int j = 0; j < NUM_FEATURES; j++)
        w[j] -= lr * grad[j] / num_samples; // atualização dos pesos
    bias -= lr * dbias / num_samples; // atualização do bias
}
double end = omp_get_wtime();
```

Metodologia

- Rodar com 1, 5, 10, 20, 40 Threads
- Rodar com 100, 1.000, 10.000, 100.000 linhas de entrada.
- Repetir 5 vezes para obter média e desvio padrão e evitar anomalias.
- Rodar normalmente e com o VTune na máquina hype (2.295 GHz 40 Cores lógicos)
- Teste usando OMP_PROC_BIND em CLOSE e SPREAD

```
1  #!/bin/bash
2  #SBATCH --job-name=logistica_vtune_csv
3  #SBATCH --partition=hype
4  #SBATCH --nodes=1
5  #SBATCH --ntasks=1
6  #SBATCH --time=04:40:00
7  #SBATCH --output=logistica_vtune_%j.out
8  #SBATCH --error=logistica_vtune_%j.err
9
10 source /home/intel/oneapi/vtune/2021.1.1/vtune-vars.sh
11 export OMP_PROC_BIND=CLOSE
12
13 PROGRAM="$HOME/PPD/Enviar/regressao4"
14 SRC="${PROGRAM}.c"
15 CSV_FILE="resultados_vtune_close_5.csv"
16
17 # Compilação
18 gcc -fopenmp -O3 -o $PROGRAM $SRC -lm
19
20 # Cabeçalho do CSV
21 echo "Threads,Samples,Repetition,Time,Loss" > $CSV_FILE
22
23 THREADS_LIST=(1 5 10 20 40)
24 NUM_ROWS=(100 1000 10000 100000)
25 REPEAT=5
26
```



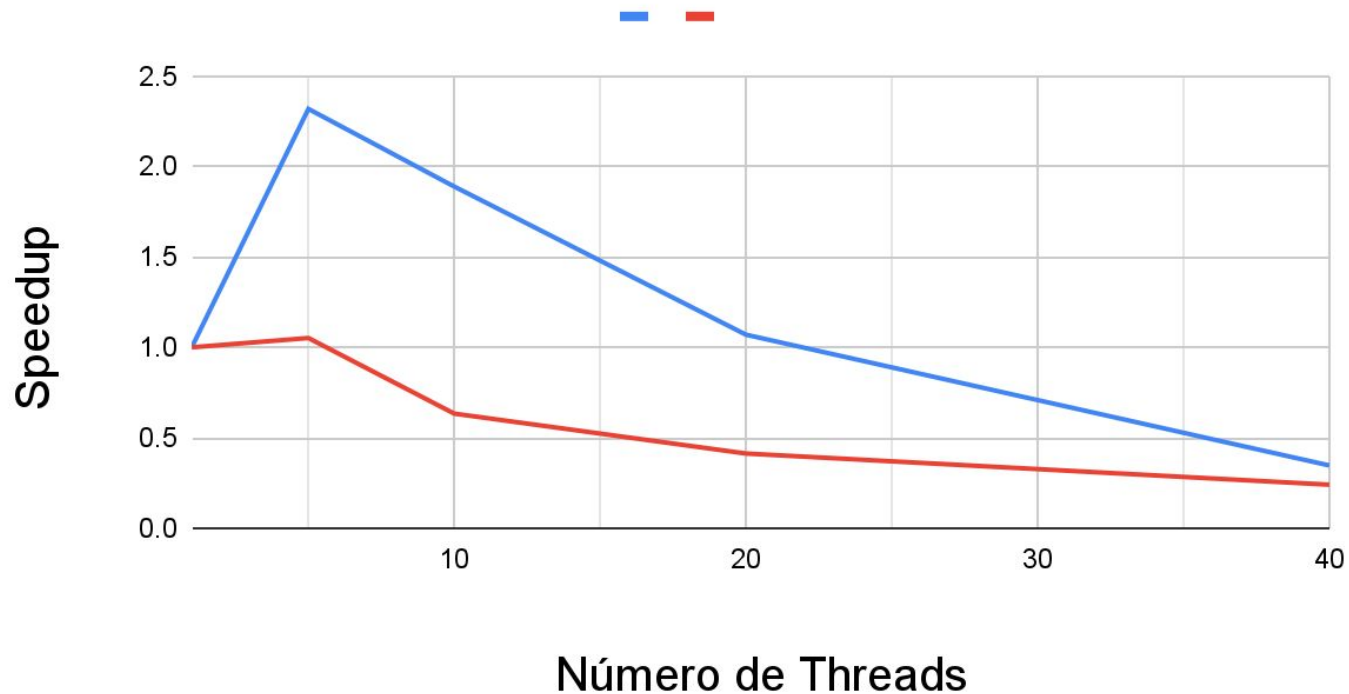
Resultados



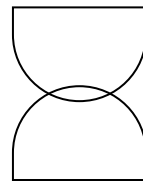
Acurácia Média de 97,25%
Perda Média de 0.033



Speedup para 100 entradas

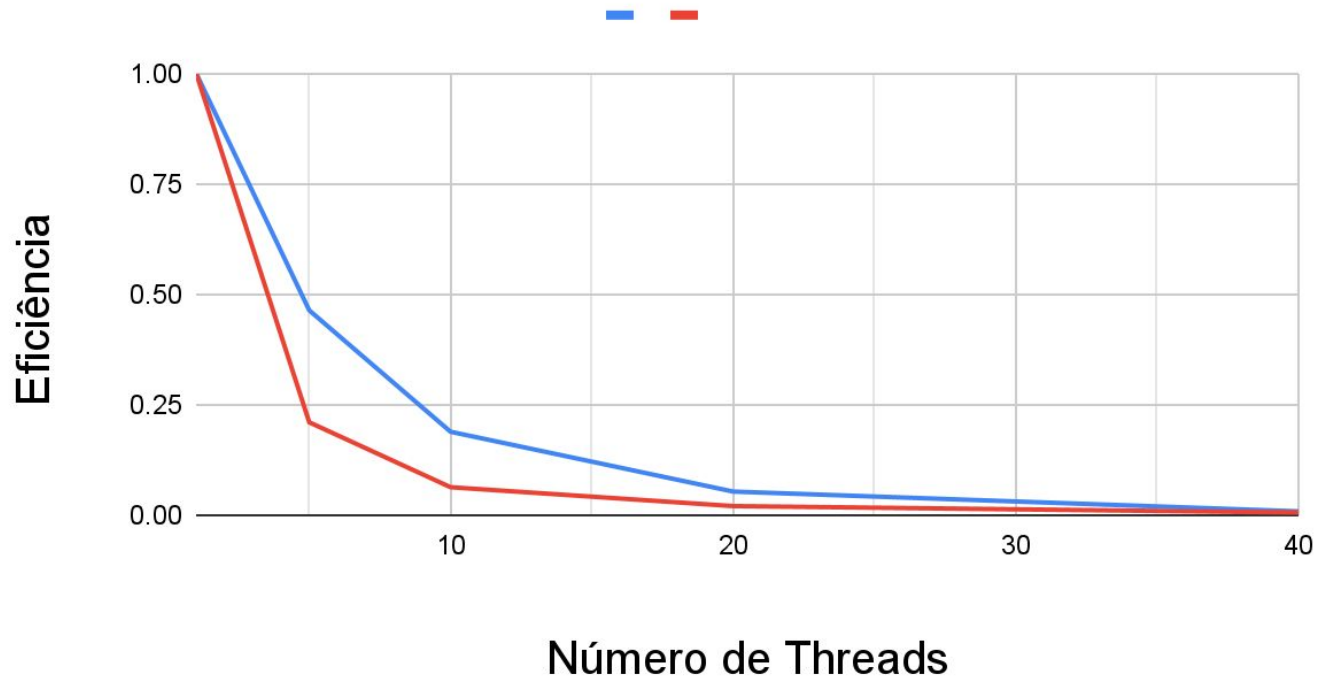


- Close
- Spread

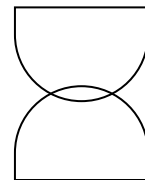




Eficiência para 100 entradas

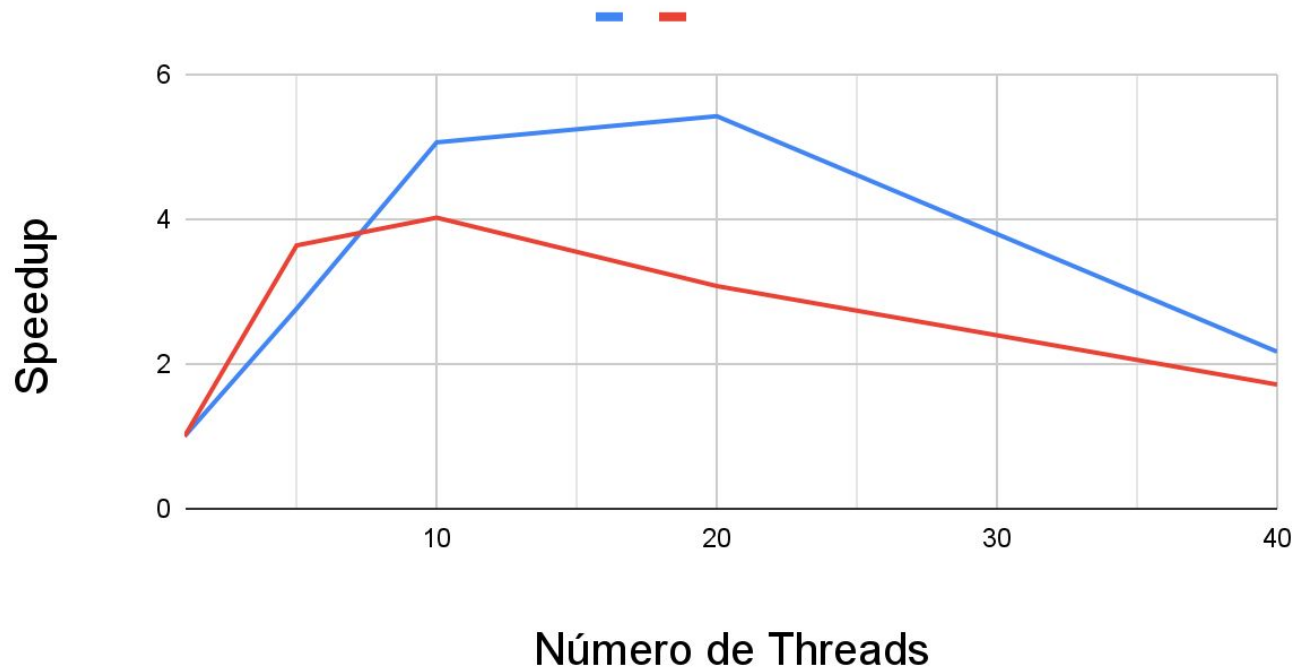


- Close
- Spread

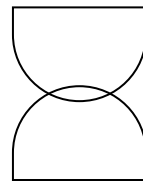




Speedup para 1000 entradas

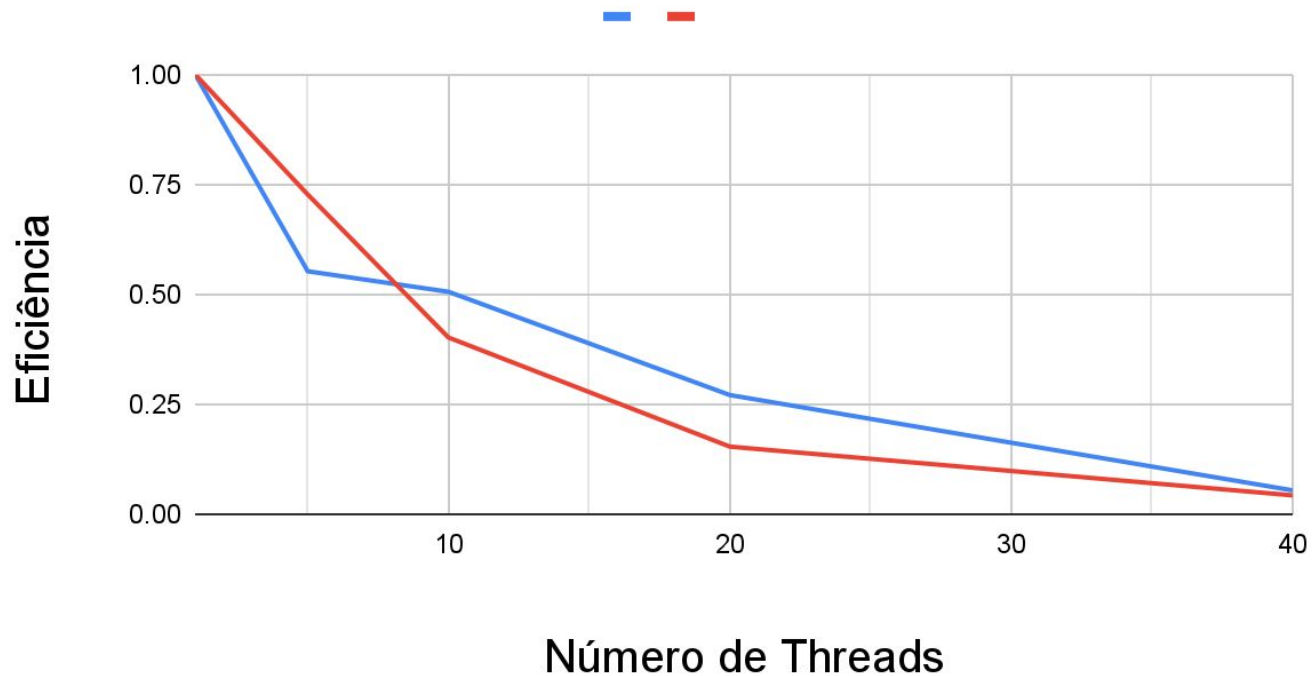


- Close
- Spread

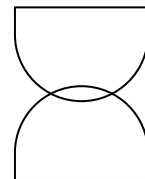




Eficiência para 1000 entradas

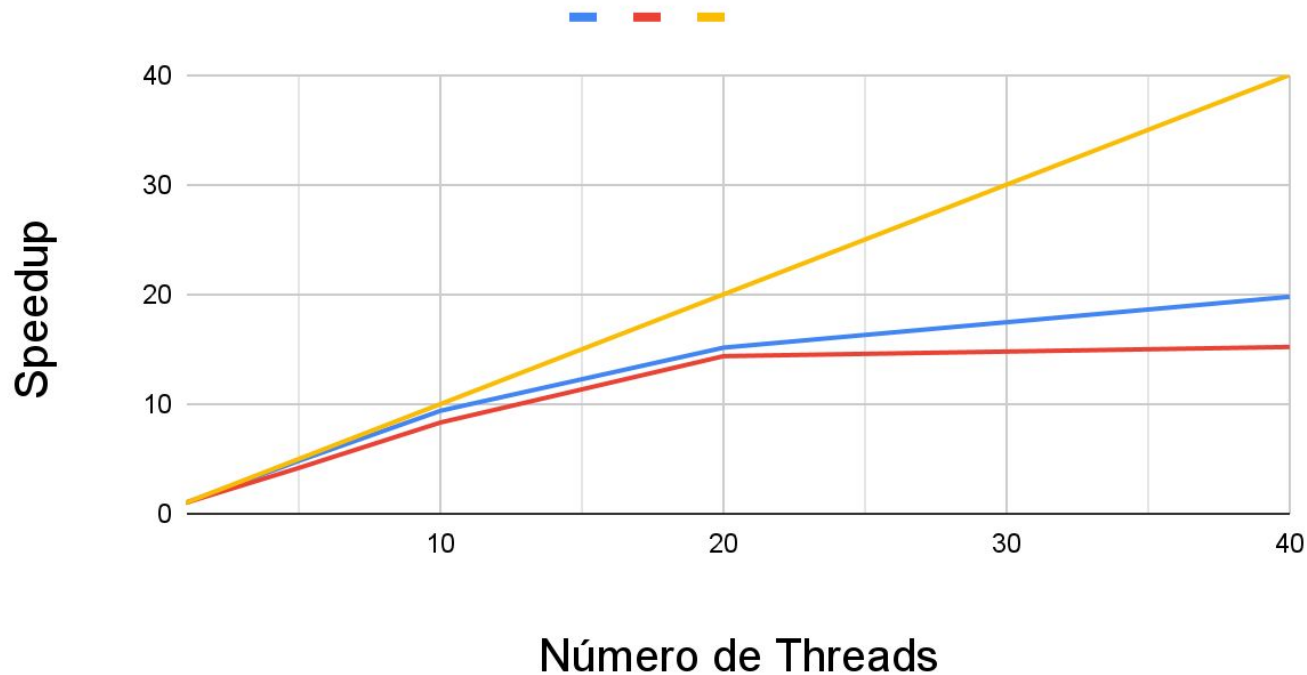


- Close
- Spread

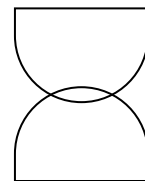




Speedup para 10000 entradas

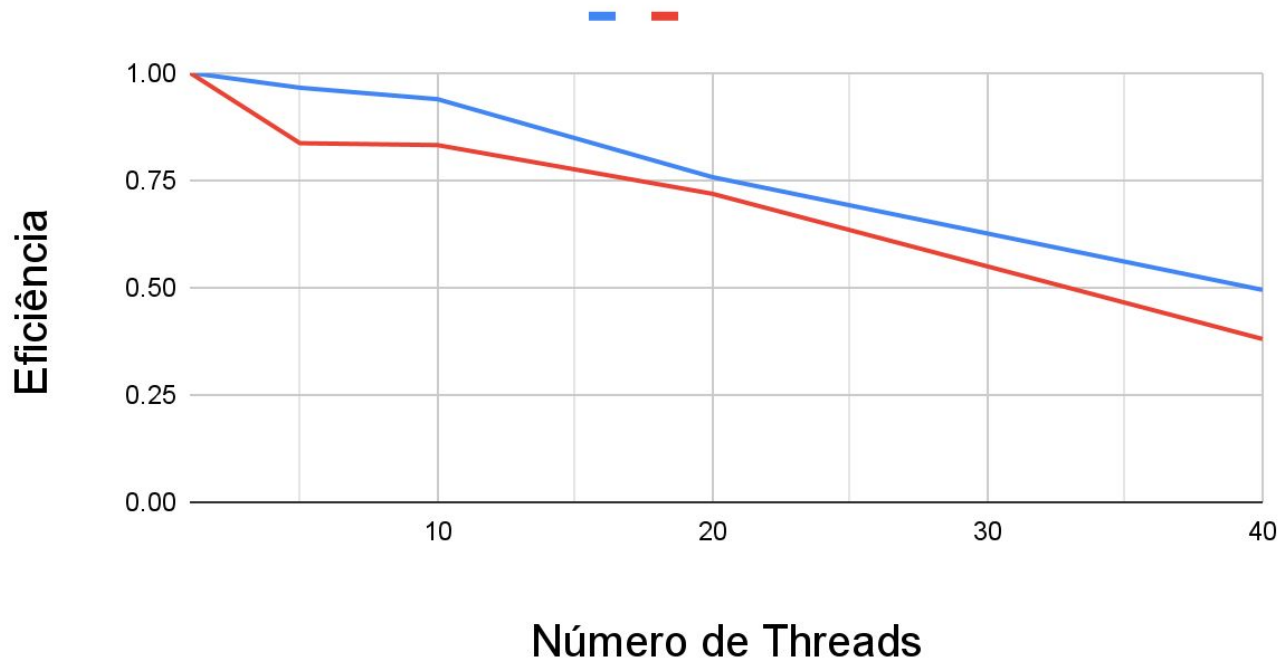


- Close
- Spread
- Speedup ideal

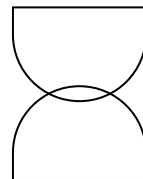




Eficiência para 10000 entradas

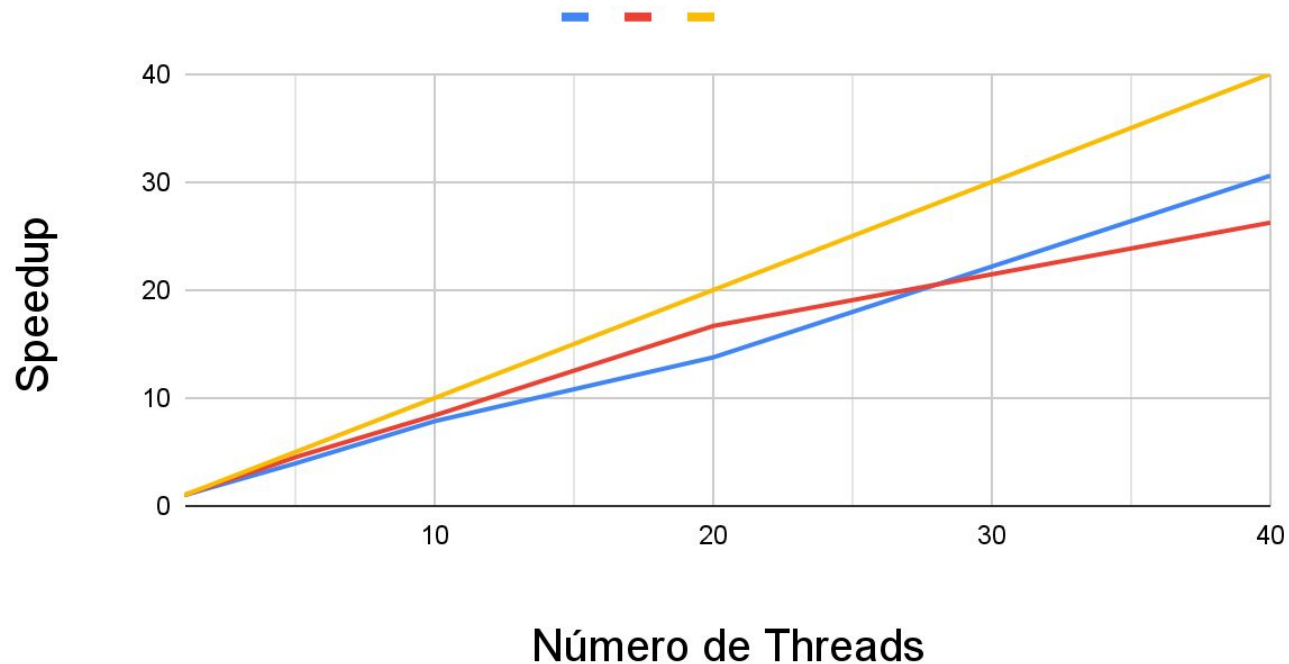


- Close
- Spread

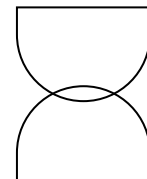




Speedup para 100000 entradas

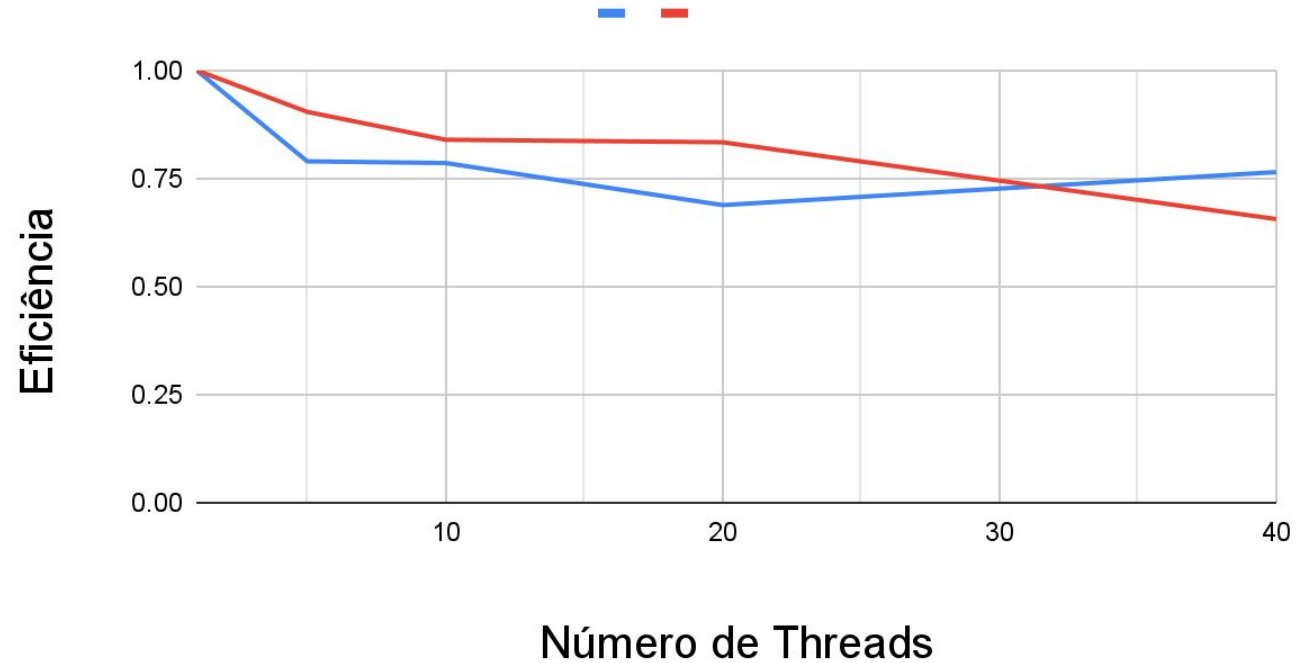


- Close
- Spread
- Speedup ideal

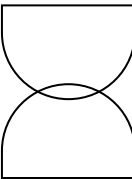


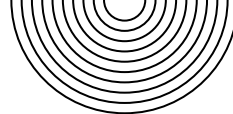


Eficiência para 100000 entradas



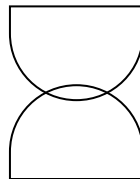
- Close
- Spread





Análise dos gráficos obtidos

- Para 100 e 1000 entradas, a eficiência é baixa devido ao overhead de troca de contexto entre threads, que representa uma parcela significativa do tempo total de execução.
- Para 100000 entradas, esse tempo se torna irrelevante e notamos uma eficiência muito boa mesmo para um grande número de threads.
- A alta eficiência se deve ao reaproveitamento das threads e à existência de uma única região paralela no nível de dados, reduzindo dependências e necessidade de sincronização, além de boa localidade de memória durante o treinamento.
- O close, em geral, tem uma eficiência ligeiramente maior, pois as threads tendem a ficar no mesmo socket do computador.



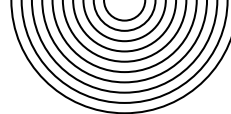


Tabela obtida (OMP_PROC_BIND = CLOSE)

Threads	1	1	5	5	10	10	20	20	40	40
Entradas	1000	100000	1000	100000	1000	100000	1000	100000	1000	100000
Tempo (s)	1.138	232.654	0.412	58.952	0.225	29.624	0.21	16.906	0.525	7.609
CPI	0.486	0.981	0.874	1.189	0.988	1.229	1.388	1.243	3.37	1.292
Phys. Core Util. (%)	4.9	4.92	12.5	15.22	19.7	24.54	30.2	48.46	73.52	95.42
Memory Bound (%)	11.52	2.78	13.6	8.04	17	6.6	30.3	6.5	44.12	7.24
Cache Bound (%)	10.3	4.6	12.74	9.04	21.8	9.86	38.08	10.1	48.3	11.34



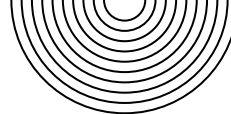
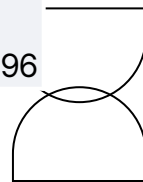
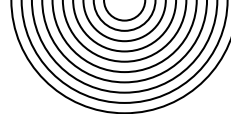


Tabela obtida (OMP_PROC_BIND = SPREAD)

Threads	1	1	5	5	10	10	20	20	40	40
Entradas	1000	100000	1000	100000	1000	100000	1000	100000	1000	100000
Tempo (s)	0.875	179.03	0.241	39.8	0.217	21.428	0.284	10.78	0.511	6.855
CPI	0.485	0.98	0.598	1.178	0.881	0.987	1.791	1.179	3.523	1.301
Phys. Core Util. (%)	4.74	4.9	19.9	24.62	31.64	49.28	77.26	97.9	79.72	96.44
Memory Bound (%)	11.02	2.72	25.68	2.76	42.4	3	60.38	2.92	43.28	7.46
Cache Bound (%)	10.3	4.52	18.34	4.84	34.56	5.14	51.78	5.26	47.28	11.96

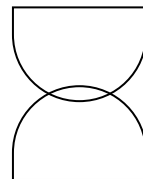


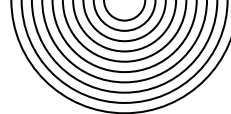


Cenário 1: Entradas Pequenas (100 e 1000)

Neste cenário, a estratégia **CLOSE** é consistentemente mais rápida.

- **Observação nos Dados:** Para 20 threads e 1000 samples, **CLOSE** leva **0.21s**, enquanto **SPREAD** leva **0.284s**. Para 40 threads, a vantagem se mantém.
- Com poucos dados, o tempo total de execução é muito curto. O volume de dados é tão pequeno que ou ele cabe inteiramente na cache de um único soquete, ou o custo de buscá-lo da RAM é mínimo. Com isso, o custo relativo da **fase de redução** se torna significativo. A alta latência da comunicação entre soquetes no **SPREAD** se torna um gargalo maior do que qualquer ganho de largura de banda. **CLOSE** vence porque sua fase de redução é extremamente eficiente.
- Note como para *Samples=100*, o **Cache Bound (%)** é altíssimo (ex: 68.48% para **CLOSE** com 20 threads). Isso mostra que a execução está totalmente focada na cache, o ambiente ideal para a estratégia **CLOSE**.





Cenário 2: Entradas Grandes (Samples = 10000 e 100000)

Neste cenário, a estratégia **SPREAD** se torna mais rápida.

- **Observação nos Dados:** Para 20 threads, a diferença é de: **CLOSE** com **16.906s** contra **SPREAD** com **10.789s**.
- Com um grande volume de dados, o tempo de execução é completamente dominado pela fase de **cálculo do gradiente**. A aplicação se torna **Memory Bound**, precisando "puxar" dados da RAM. A fase de redução, por mais que seja mais lenta no **SPREAD**, representa uma fração minúscula do tempo total. A capacidade do **SPREAD** de utilizar múltiplos controladores de memória para **maximizar a largura de banda** se torna o fator decisivo para a performance.
- Para *Samples=100000*, os indicadores de **Cache Bound (%)** e **Memory Bound (%)** são baixos. Isso não significa que a memória não é importante; pelo contrário, significa que a execução é um fluxo contínuo de dados (**streaming**) que é tão intenso que se torna o próprio gargalo, em vez de um problema de localidade de cache. A alta Utilização Física dos Núcleos com **SPREAD** (97.9% vs 48.46% para 20 threads) mostra como os núcleos estão sendo mais bem alimentados com dados, passando menos tempo ociosos, além disso cada socket contém 10 núcleos físicos, sendo utilizados exclusivamente para 1 thread.

